DESENVOLVIMENTO WEB COM .NET

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO DE AULAS PRÁTICAS

Relatório 5 – Front-end React consumindo a API com estados de requisição e roteamento

Nome: _____

RA: _____

Polo de matrícula: _____

Local da realização da Aula Prática: _____

Ano da postagem: 2026

Instituto de Ciências Exatas e Tecnologia	**Disciplina:** Desenvolvimento Web com .NET **Título da Aula:** Front-end React consumindo a API com estados de requisição e roteamento	**Relatório 5**

Unidade 5 – Integração React/Node

Objetivo: Construir uma SPA React que consuma o contrato da API, trate estados de requisição e ofereça páginas de lista e detalhes.

1. Identificação e ambiente

Repositório: https://github.com/maiaralopes-dev/projetos (branch `ccr-836bb896-v5ejij`, pasta `PortalTarefas/`); commit `28b4e8b`; projeto `src/PortalTarefas.Frontend`. **Node.js 22.23.4 (LTS) e npm 10.9.9**; Vite 8.3, React 19.3, React Router 8.5, TypeScript 7.0; testes com Vitest 5.1, Testing Library e jsdom. Execução: API em `https://localhost:7080` e `npm run dev` (Vite em `http://localhost:5173`). Variáveis em `.env.development` (versionado, sem segredo): `VITE_API_URL=` (vazio = mesma origem) e `API_PROXY_TARGET=https://localhost:7080`.

2. Resumo teórico aplicado ao projeto

Node.js e ferramenta de build. O Node não roda no navegador: executa o Vite, que serve o código em desenvolvimento com recarga instantânea, transforma TypeScript/JSX e, no build, gera arquivos estáticos otimizados (`dist/`). O Node também executa o NSwag (cliente gerado) e os testes.

Componentes, props, estado e efeitos. Componentes são funções que recebem *props* e devolvem a interface; o *estado* (`useState`) guarda o que muda. Buscar dados é um efeito colateral, então fica em `useEffect`, cujas **dependências** dizem quando refazer a busca. O hook `useRequisicao` cria um `AbortController` a cada execução e o aborta na limpeza do efeito: se o filtro muda ou a página é desmontada, a resposta antiga é **cancelada/ignorada** e não sobrescreve a nova.

CORS × proxy de desenvolvimento. CORS é uma regra que o *servidor* informa ao *navegador* sobre quais origens podem ler suas respostas. Já o proxy do Vite faz o navegador falar só com `localhost:5173`; o servidor do Vite repassa `/api` à API — não há chamada cross-origin, logo não há preflight. **Decisão:** usar o proxy em dev, porque em produção a SPA será servida pela própria API (mesma origem, unidade 8) e porque cookies de sessão (unidade 6) funcionam sem `SameSite=None`. A política CORS explícita da unidade 1 (somente `http://localhost:5173`, sem `AllowAnyOrigin`) continua como alternativa; liberar qualquer origem com credenciais nunca é aceito.

Cliente HTTP. O `fetch` só rejeita a *promise* em falha de rede; status 4xx/5xx chegam como resposta normal. O cliente gerado pelo NSwag lê o status, desserializa o JSON e **lança** o objeto tipado para respostas de erro documentadas (ex.: `ProblemDetails` no 404). A camada `api/http.ts` centraliza isso: `chamar()` nunca lança e devolve `{ok:true,dados}` ou `{ok:false,erro}`, com o erro normalizado em `rede`, `http`, `validacao` ou `cancelado`.

Estados explícitos. `idle` (antes da primeira busca), `carregando`, `sucesso`, `vazio` e `erro` são um tipo união TypeScript; a interface renderiza um ramo para cada um, o que impede telas "presas" em carregamento.

Roteamento no cliente. O React Router associa URLs a componentes sem recarregar a página: `/tarefas`, `/tarefas/nova` e `/tarefas/:id` (parâmetro lido com `useParams`). Filtros, ordenação e página ficam na **query string** (`useSearchParams`), então a URL é compartilhável e o F5 restaura a mesma tela.

3. Código-fonte e evidências

Tabela 1 – Organização do front-end

Arquivo	Responsabilidade
`src/api/gerado/portal-tarefas-client.ts`	Cliente e tipos gerados pelo NSwag a partir do OpenAPI (não editado à mão)
`src/api/http.ts`	Instância única do cliente, `chamar()`, `normalizarErro()`, simulação de atraso/falha só em DEV
`src/hooks/useRequisicao.ts`	Máquina de estados da requisição + cancelamento com `AbortController`
`src/pages/ListaTarefas.tsx`	Filtros, ordenação e paginação sincronizados com a URL
`src/pages/DetalhesTarefa.tsx`	Detalhes por id, 404 próprio e atualização otimista
`src/pages/NovaTarefa.tsx`	Formulário que exibe por campo os erros de validação da API
`vite.config.ts` / `.env.development`	Proxy `/api` → API e URL configurável por variável

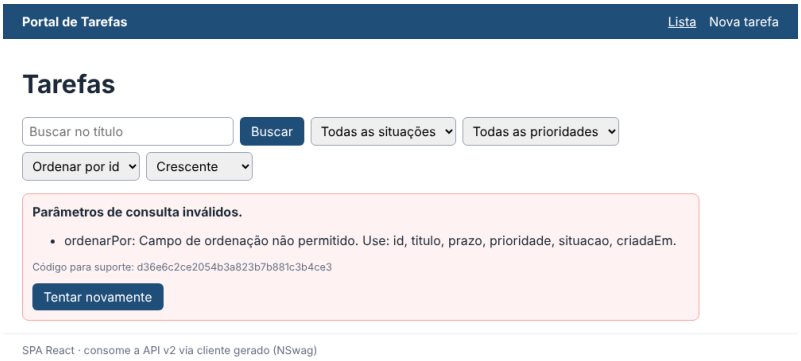


Figura 1 – Resposta HTTP 400 em Problem Details exibida com a mensagem e o código de suporte (correlationId)

Os demais estados (carregando, sucesso, vazio, erro de rede, 404, cadastro com erros e cadastro válido) estão nas Figuras B1 a B9. Navegação registrada pelo Playwright:

```
URL após filtros + próxima página: http://localhost:5173/tarefas?ordenarPor=titulo&pagina=2
Após recarregar a URL, página exibida: Página 2
Navegação por link levou a: http://localhost:5173/tarefas/3
Marcador na janela após cadastrar (sem recarregar a página): marcador
Voltar no histórico: http://localhost:5173/tarefas/nova
```

4. Testes e validações obrigatórias

Tabela 2 – Testes obrigatórios (16 testes Vitest na unidade; build de produção sem erros)

Caso exigido	Resultado obtido	Evidência
Estado inicial, carregamento e lista vazia	`idle` renderizado antes do efeito; "Carregando tarefas…" com atraso simulado; mensagem de vazio com filtro sem resultado	Teste SSR do `idle`; Fig. B1–B3
Erro de rede e resposta HTTP de erro	Falha de rede → "Falha de conexão" + tentar novamente; 400/404 → Problem Details legível	Fig. 1, B4, B6; 3 testes
Lista → detalhes por URL	Link leva a `/tarefas/3`; URL direta e voltar no histórico funcionam	Fig. B5; `navega_da_lista_*`
Filtro e paginação na URL	`?ordenarPor=titulo&pagina=2` sobrevive ao F5; "Próxima" mantém filtros	`12-navegacao.txt`; `le_filtros_*`
Cadastro válido sem recarregar a página	Navega para os detalhes com aviso; marcador em `window` preservado (sem recarga)	Fig. B8; `12-navegacao.txt`

5. Aprimoramentos

- **Testes de componentes e da camada HTTP** (Vitest + Testing Library): normalização de erros, estados, URL, cancelamento de resposta obsoleta e 404 — 16 testes.
- **Atualização otimista com reversão:** "Marcar como concluída" muda a tela na hora; se a API recusar (ex.: 409 por versão antiga), o estado anterior é restaurado e o erro é exibido (teste dedicado).
- **Acessibilidade:** o `<h1>` recebe foco a cada página; carregamento usa `role="status"` + `aria-live`, erros usam `role="alert"`, campos inválidos recebem `aria-invalid` e `aria-describedby`; link "Pular para o conteúdo"; animação respeita `prefers-reduced-motion`.
- **Simulação controlada** de atraso e falha (`?simular=atraso|falha`), disponível só em `import.meta.env.DEV`.

Referências

REACT. **Synchronizing with Effects**. react.dev, 2025. Disponível em: https://react.dev/learn/synchronizing-with-effects. Acesso em: 9 nov. 2026.

REACT. **You Might Not Need an Effect**. react.dev, 2025. Disponível em: https://react.dev/learn/you-might-not-need-an-effect. Acesso em: 9 nov. 2026.

VITE. **Server Options: server.proxy**. vite.dev, 2025. Disponível em: https://vite.dev/config/server-options. Acesso em: 9 nov. 2026.

REMIX SOFTWARE. **React Router: useSearchParams**. reactrouter.com, 2025. Disponível em: https://reactrouter.com/api/hooks/useSearchParams. Acesso em: 9 nov. 2026.

MDN WEB DOCS. **Cross-Origin Resource Sharing (CORS)**. Mozilla, 2025. Disponível em: https://developer.mozilla.org/docs/Web/HTTP/CORS. Acesso em: 9 nov. 2026.

MDN WEB DOCS. **AbortController**. Mozilla, 2025. Disponível em: https://developer.mozilla.org/docs/Web/API/AbortController. Acesso em: 9 nov. 2026.

W3C. **WAI-ARIA 1.2: status and alert roles**. 2023. Disponível em: https://www.w3.org/TR/wai-aria-1.2/. Acesso em: 9 nov. 2026.

Apêndice A – Código-fonte integral

Arquivo: `src/PortalTarefas.Frontend/src/api/http.ts` (commit `28b4e8b`)

```
// Camada única de acesso à API. Os componentes nunca chamam fetch diretamente:
// usam `chamar`, que devolve dados tipados ou um erro normalizado.
import {
  ApiException,
  PortalTarefasClient,
  type HttpValidationProblemDetails,
  type ProblemDetails,
} from './gerado/portal-tarefas-client'

export type {
  AtualizarTarefaDto,
  CriarTarefaDto,
  PaginaTarefaResumo,
  PrioridadeTarefa,
  SituacaoTarefa,
  TarefaDetalhesDto,
  TarefaResumoDto,
} from './gerado/portal-tarefas-client'

export type ErroApi =
  | { tipo: 'rede'; mensagem: string }
  | { tipo: 'http'; status: number; titulo: string; detalhe?: string; correlationId?: string }
  | { tipo: 'validacao'; titulo: string; erros: Record<string, string[]>; correlationId?: string }
  | { tipo: 'cancelado' }

export type Resultado<T> = { ok: true; dados: T } | { ok: false; erro: ErroApi }

const ehProblemDetails = (e: unknown): e is ProblemDetails & { correlationId?: string } =>
  typeof e === 'object' && e !== null && 'status' in e && 'title' in e

/** Converte qualquer falha (rede, cancelamento, status HTTP, Problem Details) em um formato único. */
export function normalizarErro(e: unknown): ErroApi {
  if (e instanceof DOMException && e.name === 'AbortError') return { tipo: 'cancelado' }
  if (e instanceof TypeError) return { tipo: 'rede', mensagem: 'Não foi possível conectar à API. Verifique sua
conexão.' }
  if (ehProblemDetails(e)) {
    const pd = e as HttpValidationProblemDetails & { correlationId?: string }
    if (pd.status === 400 && pd.errors)
      return { tipo: 'validacao', titulo: pd.title ?? 'Dados inválidos.', erros: pd.errors, correlationId:
pd.correlationId }
    return { tipo: 'http', status: pd.status ?? 0, titulo: pd.title ?? 'Erro na API.', detalhe: pd.detail,
correlationId: pd.correlationId }
  }
  if (e instanceof ApiException) {
    // Status não documentado no contrato: tenta ler um Problem Details do corpo.
    try {
      const corpo = JSON.parse(e.response)
      if (ehProblemDetails(corpo)) return normalizarErro(corpo)
    } catch { /* corpo não é JSON */ }
    return { tipo: 'http', status: e.status, titulo: `Erro ${e.status} ao chamar a API.` }
  }
  return { tipo: 'http', status: 0, titulo: 'Erro inesperado.' }
}

// Em desenvolvimento, ?simular=atraso ou ?simular=falha na URL permite observar os estados de carregamento e
erro.
async function fetchComSimulacao(input: RequestInfo | URL, init?: RequestInit): Promise<Response> {
  if (import.meta.env.DEV && typeof window !== 'undefined') {
    const simular = new URLSearchParams(window.location.search).get('simular')
    if (simular === 'atraso' || simular === 'falha') {
      await new Promise((ok, falha) => {
        const t = setTimeout(ok, 1500)
        init?.signal?.addEventListener('abort', () => { clearTimeout(t); falha(new DOMException('cancelado',
'AbortError')) })
      })
    }
    if (simular === 'falha') throw new TypeError('Failed to fetch (simulado)')
  }
  return fetch(input, { ...init, credentials: 'same-origin' })
}

export const cliente = new PortalTarefasClient(import.meta.env.VITE_API_URL ?? '', { fetch: fetchComSimulacao })

/** Executa uma operação do cliente gerado e devolve dados tipados ou erro normalizado (nunca lança). */
export async function chamar<T>(
  operacao: (c: PortalTarefasClient, signal: AbortSignal) => Promise<T>,
  signal: AbortSignal = new AbortController().signal,
): Promise<Resultado<T>> {
  try {
    return { ok: true, dados: await operacao(cliente, signal) }
  } catch (e) {
    return { ok: false, erro: signal.aborted ? { tipo: 'cancelado' } : normalizarErro(e) }
  }
}
```

Arquivo: `src/PortalTarefas.Frontend/src/hooks/useRequisicao.ts` (commit `28b4e8b`)

```
import { useCallback, useEffect, useState, type DependencyList } from 'react'
import { chamar, type ErroApi } from '../api/http'
import type { PortalTarefasClient } from '../api/gerado/portal-tarefas-client'

/** Estados explícitos de uma requisição de leitura. */
export type EstadoRequisicao<T> =
  | { fase: 'idle' }
  | { fase: 'carregando' }
  | { fase: 'sucesso'; dados: T }
  | { fase: 'vazio'; dados: T }
  | { fase: 'erro'; erro: ErroApi }

/**
 * Executa a operação sempre que as dependências mudam. A requisição anterior é cancelada
 * (AbortController), então uma resposta obsoleta nunca sobrescreve a mais recente.
 */
export function useRequisicao<T>(
  operacao: (c: PortalTarefasClient, signal: AbortSignal) => Promise<T>,
  deps: DependencyList,
  ehVazio: (dados: T) => boolean = () => false,
) {
  const [estado, setEstado] = useState<EstadoRequisicao<T>>({ fase: 'idle' })
  const [tentativa, setTentativa] = useState(0)

  useEffect(() => {
    const controle = new AbortController()
    setEstado({ fase: 'carregando' })
    chamar(operacao, controle.signal).then((r) => {
      if (controle.signal.aborted) return            // componente desmontado ou filtros mudaram
      if (!r.ok) setEstado(r.erro.tipo === 'cancelado' ? { fase: 'idle' } : { fase: 'erro', erro: r.erro })
      else setEstado(ehVazio(r.dados) ? { fase: 'vazio', dados: r.dados } : { fase: 'sucesso', dados: r.dados })
    })
    return () => controle.abort()
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [...deps, tentativa])

  const recarregar = useCallback(() => setTentativa((t) => t + 1), [])
  return { estado, setEstado, recarregar }
}
```

Arquivo: `src/PortalTarefas.Frontend/src/pages/ListaTarefas.tsx` (commit `28b4e8b`)

```
import { Link, useSearchParams } from 'react-router'
import type { PaginaTarefaResumo, PrioridadeTarefa, SituacaoTarefa } from '../api/http'
import { Carregando, MensagemErro, Selo, TituloPagina, formatarData, rotuloSituacao } from '../components/Comuns'
import { useRequisicao } from '../hooks/useRequisicao'

const TAMANHO = 5

/** Lista com filtros, ordenação e paginação sincronizados com a URL (?situacao=&pagina=...). */
export function ListaTarefas() {
  const [params, setParams] = useSearchParams()
  const pagina = Number(params.get('pagina') ?? '1')
  const situacao = (params.get('situacao') || undefined) as SituacaoTarefa | undefined
  const prioridade = (params.get('prioridade') || undefined) as PrioridadeTarefa | undefined
  const busca = params.get('busca') || undefined
  const ordenarPor = params.get('ordenarPor') || undefined
  const direcao = params.get('direcao') || undefined

  const { estado, recarregar } = useRequisicao<PaginaTarefaResumo>(
    (c, signal) => c.listarTarefas(pagina, TAMANHO, ordenarPor, direcao, situacao, prioridade, busca, undefined,
signal),
    [pagina, situacao, prioridade, busca, ordenarPor, direcao],
    (p) => p.itens.length === 0,
  )

  /** Altera um filtro na URL; qualquer filtro novo volta para a página 1. */
  const definir = (chave: string, valor: string) => {
    const novos = new URLSearchParams(params)
    if (valor) novos.set(chave, valor); else novos.delete(chave)
    if (chave !== 'pagina') novos.delete('pagina')
    setParams(novos)
  }

  return (
    <>
      <TituloPagina>Tarefas</TituloPagina>
      <form className="filtros" role="search" onSubmit={(e) => { e.preventDefault(); definir('busca', new
FormData(e.currentTarget).get('busca') as string) }}>
        <input name="busca" defaultValue={busca} key={busca} placeholder="Buscar no título" aria-label="Buscar no
título" />
        <button type="submit">Buscar</button>
        <select aria-label="Situação" value={situacao ?? ''} onChange={(e) => definir('situacao',
e.target.value)}>
          <option value="">Todas as situações</option>
          <option value="Pendente">Pendente</option><option value="EmAndamento">Em andamento</option><option
value="Concluida">Concluída</option>
        </select>
        <select aria-label="Prioridade" value={prioridade ?? ''} onChange={(e) => definir('prioridade',
```

```
e.target.value)}>
          <option value="">Todas as prioridades</option>
          <option value="Baixa">Baixa</option><option value="Media">Média</option><option
value="Alta">Alta</option>
        </select>
        <select aria-label="Ordenar por" value={ordenarPor ?? ''} onChange={(e) => definir('ordenarPor',
e.target.value)}>
          <option value="">Ordenar por id</option><option value="titulo">Título</option><option
value="prazo">Prazo</option><option value="prioridade">Prioridade</option>
        </select>
        <select aria-label="Direção" value={direcao ?? ''} onChange={(e) => definir('direcao', e.target.value)}>
          <option value="">Crescente</option><option value="desc">Decrescente</option>
        </select>
      </form>

      {estado.fase === 'idle' && <p className="estado" role="status">Pronto para consultar.</p>}
      {estado.fase === 'carregando' && <Carregando texto="Carregando tarefas…" />}
      {estado.fase === 'erro' && <MensagemErro erro={estado.erro} onTentarNovamente={recarregar} />}
      {estado.fase === 'vazio' && (
        <p className="estado vazio" role="status">Nenhuma tarefa encontrada com esses filtros.</p>
      )}
      {estado.fase === 'sucesso' && (
        <>
          <p className="resumo" role="status">{estado.dados.totalItens} tarefa(s) · página {estado.dados.pagina}
de {estado.dados.totalPaginas}</p>
          <ul className="lista">
            {estado.dados.itens.map((t) => (
              <li key={t.id} className={`cartao ${t.vencida ? 'vencida' : ''}`}>
                <h3><Link to={`/tarefas/${t.id}`}>#{t.id} · {t.titulo}</Link></h3>
                <Selo classe={`situacao-${t.situacao}`}>{rotuloSituacao[t.situacao]}</Selo>
                <Selo classe={`prioridade-${t.prioridade}`}>Prioridade {t.prioridade}</Selo>
                <span className="prazo">Prazo: {formatarData(t.prazo)}</span>
                {t.vencida && <strong className="alerta">Vencida</strong>}
              </li>
            ))}
          </ul>
          <nav className="paginacao" aria-label="Paginação">
            <button type="button" disabled={pagina <= 1} onClick={() => definir('pagina', String(pagina -
1))}>Anterior</button>
            <span>Página {pagina}</span>
            <button type="button" disabled={pagina >= estado.dados.totalPaginas} onClick={() => definir('pagina',
String(pagina + 1))}>Próxima</button>
          </nav>
        </>
      )}
    </>
  )
}
```

Arquivo: `src/PortalTarefas.Frontend/src/pages/DetalhesTarefa.tsx` (commit `28b4e8b`)

```
import { useState } from 'react'
import { Link, useLocation, useParams } from 'react-router'
import { chamar, type ErroApi, type TarefaDetalhesDto } from '../api/http'
import { Carregando, MensagemErro, Selo, TituloPagina, formatarData, rotuloSituacao } from '../components/Comuns'
import { useRequisicao } from '../hooks/useRequisicao'

export function DetalhesTarefa() {
  const { id } = useParams()
  const idNumerico = Number(id)
  const mensagem = (useLocation().state as { mensagem?: string } | null)?.mensagem
  const { estado, setEstado, recarregar } = useRequisicao<TarefaDetalhesDto>((c, s) => c.obterTarefa(idNumerico,
s), [idNumerico])
  const [erroAcao, setErroAcao] = useState<ErroApi | null>(null)

  if (!Number.isInteger(idNumerico) || idNumerico <= 0)
    return <><TituloPagina>Endereço inválido</TituloPagina><p>O identificador “{id}” não é válido.</p></>

  /** Atualização otimista: a tela muda na hora; se a API recusar, o estado anterior é restaurado. */
  async function concluir(tarefa: TarefaDetalhesDto) {
    setErroAcao(null)
    setEstado({ fase: 'sucesso', dados: { ...tarefa, situacao: 'Concluida' } })
    const r = await chamar((c, s) => c.atualizarTarefa(tarefa.id, {
      titulo: tarefa.titulo, descricao: tarefa.descricao, situacao: 'Concluida',
      prioridade: tarefa.prioridade, prazo: tarefa.prazo, versao: tarefa.versao,
    }, s))
    if (r.ok) setEstado({ fase: 'sucesso', dados: r.dados })
    else { setEstado({ fase: 'sucesso', dados: tarefa }); setErroAcao(r.erro) }   // reversão
  }

  return (
    <>
      <TituloPagina>Detalhes da tarefa</TituloPagina>
      {mensagem && <p className="aviso" role="status">{mensagem}</p>}
      {(estado.fase === 'idle' || estado.fase === 'carregando') && <Carregando texto="Carregando tarefa…" />}
      {estado.fase === 'erro' && estado.erro.tipo === 'http' && estado.erro.status === 404 && (
        <div className="estado erro" role="alert"><strong>Tarefa não encontrada</strong><p>Não existe tarefa com
o identificador {id}.</p></div>
      )}
      {estado.fase === 'erro' && !(estado.erro.tipo === 'http' && estado.erro.status === 404) && (
        <MensagemErro erro={estado.erro} onTentarNovamente={recarregar} />
```

```
      )}
      {estado.fase === 'sucesso' && (
        <article className="cartao">
          <h2>#{estado.dados.id} · {estado.dados.titulo}</h2>
          <p>
            <Selo classe={`situacao-${estado.dados.situacao}`}>{rotuloSituacao[estado.dados.situacao]}</Selo>
            <Selo classe={`prioridade-${estado.dados.prioridade}`}>Prioridade {estado.dados.prioridade}</Selo>
          </p>
          <dl>
            <dt>Descrição</dt><dd>{estado.dados.descricao ?? '(sem descrição)'}</dd>
            <dt>Prazo</dt><dd>{formatarData(estado.dados.prazo)}</dd>
            <dt>Criada em</dt><dd>{new Date(estado.dados.criadaEm).toLocaleString('pt-BR')}</dd>
          </dl>
          {estado.dados.situacao !== 'Concluida' && (
            <button type="button" onClick={() => concluir(estado.dados)}>Marcar como concluída</button>
          )}
          {erroAcao && <MensagemErro erro={erroAcao} />}
        </article>
      )}
      <p><Link to="/tarefas">← Voltar à lista</Link></p>
    </>
  )
}
```

Arquivo: `src/PortalTarefas.Frontend/src/pages/NovaTarefa.tsx` (commit `28b4e8b`)

```
import { useState, type FormEvent } from 'react'
import { useNavigate } from 'react-router'
import { chamar, type CriarTarefaDto, type ErroApi, type PrioridadeTarefa } from '../api/http'
import { MensagemErro, TituloPagina } from '../components/Comuns'

/** Formulário de cadastro. Erros de validação vêm da API (Problem Details 400) e são exibidos por campo. */
export function NovaTarefa() {
  const navegar = useNavigate()
  const [enviando, setEnviando] = useState(false)
  const [erro, setErro] = useState<ErroApi | null>(null)

  const errosCampo = erro?.tipo === 'validacao'
    ? Object.fromEntries(Object.entries(erro.erros).map(([k, v]) => [k.toLowerCase(), v.join(' ')]))
    : {}

  async function enviar(e: FormEvent<HTMLFormElement>) {
    e.preventDefault()
    const f = new FormData(e.currentTarget)
    const dto: CriarTarefaDto = {
      titulo: String(f.get('titulo') ?? ''),
      descricao: String(f.get('descricao') ?? '') || undefined,
      prioridade: f.get('prioridade') as PrioridadeTarefa,
      prazo: String(f.get('prazo') ?? '') || undefined,
    }
    setEnviando(true); setErro(null)
    const r = await chamar((c, s) => c.criarTarefa(dto, s))
    setEnviando(false)
    if (r.ok) navegar(`/tarefas/${r.dados.id}`, { state: { mensagem: `Tarefa “${r.dados.titulo}” cadastrada.` } })
    else setErro(r.erro)
  }

  const campo = (nome: string) => ({ 'aria-invalid': !!errosCampo[nome], 'aria-describedby': errosCampo[nome] ? `erro-${nome}` : undefined })
  const msg = (nome: string) => errosCampo[nome] && <span id={`erro-${nome}`} className="erro-campo">{errosCampo[nome]}</span>

  return (
    <>
      <TituloPagina>Nova tarefa</TituloPagina>
      <form onSubmit={enviar} noValidate className="formulario">
        <label>Título<input name="titulo" {...campo('titulo')} /></label>{msg('titulo')}
        <label>Descrição<textarea name="descricao" rows={3} {...campo('descricao')} /></label>{msg('descricao')}
        <div className="linha">
          <label>Prioridade
            <select name="prioridade" defaultValue="Media"><option value="Baixa">Baixa</option><option value="Media">Média</option><option value="Alta">Alta</option></select>
          </label>
          <label>Prazo<input name="prazo" type="date" {...campo('prazo')} /></label>
        </div>
        {msg('prazo')}
        {erro && erro.tipo !== 'validacao' && <MensagemErro erro={erro} />}
        {erro?.tipo === 'validacao' && <p className="erro-campo" role="alert">Corrija os campos destacados.</p>}
        <button type="submit" disabled={enviando}>{enviando ? 'Salvando…' : 'Cadastrar'}</button>
      </form>
    </>
  )
}
```

Arquivo: `src/PortalTarefas.Frontend/src/components/Comuns.tsx` (commit `28b4e8b`)

```
import { useEffect, useRef, type ReactNode } from 'react'
import type { ErroApi } from '../api/http'
```

```
/** Título da página: recebe o foco ao montar, para leitores de tela anunciarem a navegação. */
export function TituloPagina({ children }: { children: ReactNode }) {
  const ref = useRef<HTMLHeadingElement>(null)
  useEffect(() => ref.current?.focus(), [])
  return <h1 ref={ref} tabIndex={-1}>{children}</h1>
}

export function Carregando({ texto = 'Carregando…' }: { texto?: string }) {
  return <p className="estado carregando" role="status" aria-live="polite"><span className="spinner" aria-
hidden="true" />{texto}</p>
}

export function MensagemErro({ erro, onTentarNovamente }: { erro: ErroApi; onTentarNovamente?: () => void }) {
  if (erro.tipo === 'cancelado') return null
  const titulo = erro.tipo === 'rede' ? 'Falha de conexão' : erro.titulo
  return (
    <div className="estado erro" role="alert">
      <strong>{titulo}</strong>
      {erro.tipo === 'rede' && <p>{erro.mensagem}</p>}
      {erro.tipo === 'http' && <p>{erro.detalhe ?? `A API respondeu com status ${erro.status}.`}</p>}
      {erro.tipo === 'validacao' && (
        <ul>{Object.entries(erro.erros).map(([campo, msgs]) => <li key={campo}>{campo}: {msgs.join('
')}</li>)}</ul>
      )}
      {'correlationId' in erro && erro.correlationId && <p className="corr">Código para suporte:
{erro.correlationId}</p>}
      {onTentarNovamente && <button type="button" onClick={onTentarNovamente}>Tentar novamente</button>}
    </div>
  )
}

export function Selo({ children, classe }: { children: ReactNode; classe?: string }) {
  return <span className={`selo ${classe ?? ''}`}>{children}</span>
}

export const formatarData = (iso?: string) => (iso ? iso.split('-').reverse().join('/') : '—')
export const rotuloSituacao: Record<string, string> = { Pendente: 'Pendente', EmAndamento: 'Em andamento',
Concluida: 'Concluída' }
```

Arquivo: `src/PortalTarefas.Frontend/src/App.tsx` (commit `28b4e8b`)

```
import { NavLink, Navigate, Route, Routes } from 'react-router'
import { TituloPagina } from './components/Comuns'
import { DetalhesTarefa } from './pages/DetalhesTarefa'
import { ListaTarefas } from './pages/ListaTarefas'
import { NovaTarefa } from './pages/NovaTarefa'

export default function App() {
  return (
    <>
      <a className="pular" href="#conteudo">Pular para o conteúdo</a>
      <header className="topo">
        <span className="marca">Portal de Tarefas</span>
        <nav aria-label="Principal">
          <NavLink to="/tarefas" end>Lista</NavLink>
          <NavLink to="/tarefas/nova">Nova tarefa</NavLink>
        </nav>
      </header>
      <main id="conteudo">
        <Routes>
          <Route path="/" element={<Navigate to="/tarefas" replace />} />
          <Route path="/tarefas" element={<ListaTarefas />} />
          <Route path="/tarefas/nova" element={<NovaTarefa />} />
          <Route path="/tarefas/:id" element={<DetalhesTarefa />} />
          <Route path="*" element={<><TituloPagina>Página não encontrada</TituloPagina><p>Verifique o
endereço.</p></>} />
        </Routes>
      </main>
      <footer>SPA React · consome a API v2 via cliente gerado (NSwag)</footer>
    </>
  )
}
```

Arquivo: `src/PortalTarefas.Frontend/vite.config.ts` (commit `28b4e8b`)

```
/// <reference types="vitest/config" />
import react from '@vitejs/plugin-react'
import { defineConfig, loadEnv } from 'vite'

export default defineConfig(({ mode }) => {
  const env = loadEnv(mode, process.cwd(), '')
  const alvo = env.API_PROXY_TARGET || 'https://localhost:7080'
  return {
    plugins: [react()],
    server: {
      port: 5173,
      strictPort: true,
      // Proxy de desenvolvimento: as chamadas /api saem do servidor do Vite, não do navegador,
      // então não há requisição cross-origin (nem preflight CORS) e cookies ficam na mesma origem.
      proxy: {
```

```
        '/api': { target: alvo, secure: false },        // secure:false só porque o certificado de dev é
autoassinado
        '/openapi': { target: alvo, secure: false },
      },
    },
    test: {
      environment: 'jsdom',
      setupFiles: ['./src/testes/setup.ts'],
      css: false,
    },
  }
})
```

Arquivo: `src/PortalTarefas.Frontend/.env.development` (commit `28b4e8b`)

```
# Desenvolvimento: o navegador fala só com o Vite (http://localhost:5173), que repassa /api para a API.
# Sem segredos aqui — este arquivo é versionado.
VITE_API_URL=
API_PROXY_TARGET=https://localhost:7080
```

Apêndice B – Capturas e saídas

Proxy de desenvolvimento, testes e build de produção

```
$ curl http://localhost:5173/api/v2/tarefas?tamanho=1   # via proxy do Vite (mesma origem para o
navegador)
HTTP/1.1 200 OK
content-type: application/json; charset=utf-8
x-total-count: 9


$ npm test
 RUN  v5.1.0 /home/user/projetos/PortalTarefas/src/PortalTarefas.Frontend


 Test Files  2 passed (2)
      Tests  16 passed (16)
   Start at  01:36:43
   Duration  888ms (environment 56%, tests 22%, setup 14%, import 5%, transform 3%)


$ npm run build
computing gzip size...
dist/index.html                   0.47 kB │ gzip:  0.30 kB
dist/assets/index-DE8CcRa0.css    2.47 kB │ gzip:  1.04 kB
dist/assets/index-KaC6dK8A.js   275.24 kB │ gzip: 86.19 kB

✓ built in 153ms
```



Figura B1 – Estado carregando (atraso simulado)

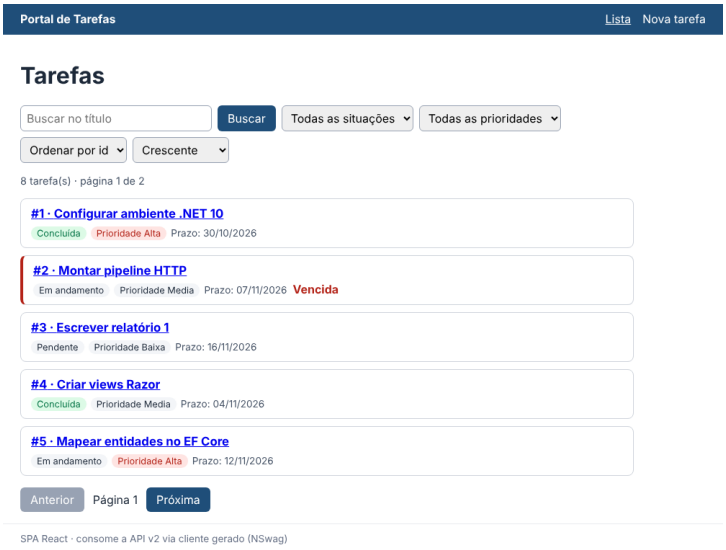


Figura B2 – Sucesso: lista paginada

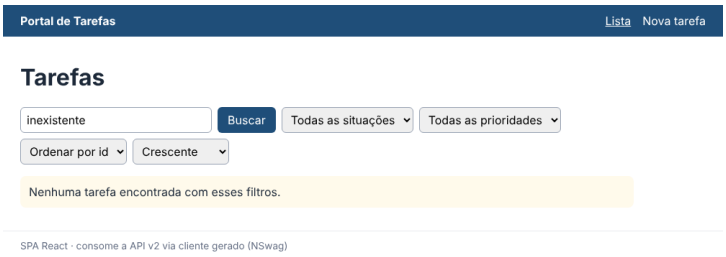


Figura B3 – Lista vazia para um filtro sem resultados

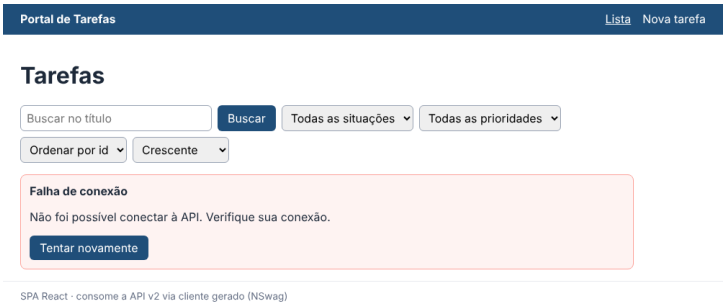


Figura B4 – Erro de rede com opção de tentar novamente

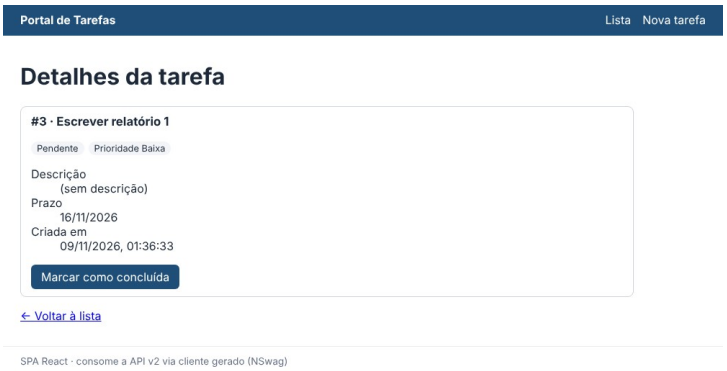


Figura B5 – Detalhes acessados por link/URL



Figura B6 – Detalhes de identificador inexistente (404)

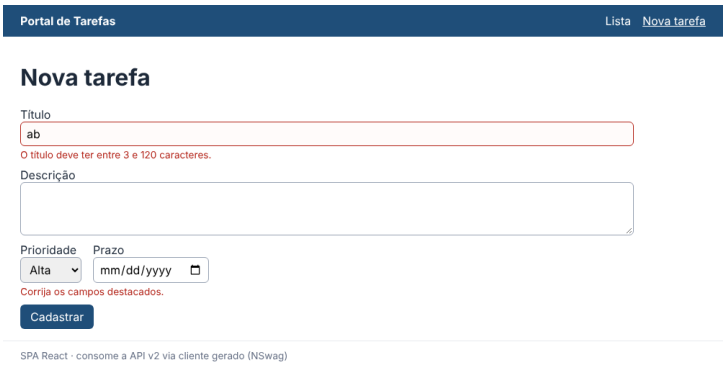


Figura B7 – Cadastro com erros de validação devolvidos pela API

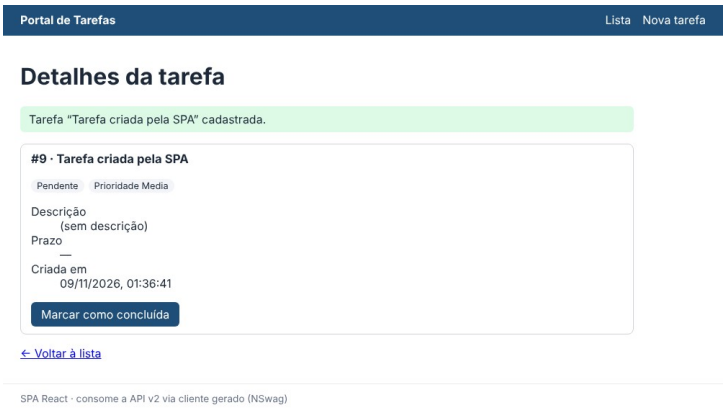


Figura B8 – Cadastro válido: navegação para os detalhes sem recarregar a página



Figura B9 – Atualização otimista concluída